

High Efficiency Synchronous Binary Counter with Minimal Clock Period Using Optimized Parallel Architecture

SOFTWARE CODE

```
module tff(en,clk,t,q);
input en,clk,t;
output reg q;
always@(posedge clk)
begin
if(en)
q<=~t;
else
q<=q;
end
endmodule
```

```
module tffwen(en,clk,q);
input en, clk;
output reg q;
initial q=1'b0;
always@(posedge clk)
begin
//if(en)
q<=en^q;
//else//q<=1'b0;
end
endmodule
```

```

module conv_8bsync(cnt,clk,q);
input cnt, clk;
output [7:0]q;
wire [7:0]q1;
wire y1,y2,y3,y4,y5,y6;
tffwen u1(cnt,clk,q1[0]);
tff u2(cnt,clk,q1[0], q1[1]);
assign y1 = q1[0] & q1[1];
tff u3(cnt,clk,y1,q1[2]);
assign y2 = y1 & q1[2];
tff u4(cnt,clk,y2,q1[3]);
assign y3 = y2 & q1[3];
tff u5(cnt,clk,y3,q1[4]);
assign y4 = y3 & q1[4];
tff u6(cnt,clk,y4,q1[5]);
assign y5 = y4 & q1[5];
tff u7(cnt,clk,y5,q1[6]);
assign y6 = y5 & q1[6];
tff u8(cnt,clk,y6,q1[7]);
assign q = q1;
endmodule

```

```

module sync_cnt8_BCP(cnt,clk,q);
input cnt,clk;
output [7:0]q;
wire [7:0]q1;
wire q2,q3,q4,q5,q6,q7;
tffwen u1(cnt,clk,q1[0]);

```

```

tffwen u2(q1[0],clk,q1[1]);
assign q2 = q1[1] & q1[0];
tffwen u3(q2,clk,q1[2]);
assign q3 = q2 & q1[2];
tffwen u4(q3,clk,q1[3]);
assign q4 = q3 & q1[3];
tffwen u5(q4,clk,q1[4]);
assign q5 = q4 & q1[4];
tffwen u6(q5,clk,q1[5]);
assign q6 = q5 & q1[5];
tffwen u7(q6,clk,q1[6]);
assign q7 = q6 & q1[6];
tffwen u8(q7,clk,q1[7]);
assign q = q1;
endmodule

```

```

module ring_cnt_2b(cnt,clk,q);
input cnt,clk;
output reg [1:0]q;
initial q<=2'b01;
always@(posedge clk)
begin
if (cnt)
begin
q[1]<=q[0];
q[0]<=q[1];
end
else

```

```
q <=2'b0;  
end  
endmodule
```

```
module ring_cnt_4b(cnt,clk,q);  
input cnt,clk;  
output reg [3:0]q;  
initial q<=4'b0001;  
always@(posedge clk)  
begin  
if (cnt)  
begin  
q[1]<=q[0];  
q[2]<=q[1];  
q[3]<=q[2];  
q[0]<=q[3];  
end  
else  
q <=4'b0;  
end  
endmodule
```

```
module ring_cnt_8b(cnt,clk,q);  
input cnt,clk;  
output reg [7:0]q;  
initial q<=8'b00000001;  
always@(posedge clk)  
begin
```

```
if (cnt)
begin
q[1]<=q[0];
q[2]<=q[1];
q[3]<=q[2];
q[4]<=q[3];
q[5]<=q[4];
q[6]<=q[5];
q[7]<=q[6];
q[0]<=q[7];
end
else
q <=8'b0;
end
endmodule
```

```
module ring_cnt_16b(cnt,clk,q);
input cnt,clk;
output reg [15:0]q;
initial q<=16'd1;
always@(posedge clk)
begin
if (cnt)
begin
q[1]<=q[0];
q[2]<=q[1];
q[3]<=q[2];
q[4]<=q[3];
```

```
q[5]<=q[4];
q[6]<=q[5];
q[7]<=q[6];
q[8]<=q[7];
q[9]<=q[8];
q[10]<=q[9];
q[11]<=q[10];
q[12]<=q[11];
q[13]<=q[12];
q[14]<=q[13];
q[15]<=q[14];
q[0]<=q[15];
end
else
q <=64'b0;
end
endmodule
```

```
module ring_cnt_32b(cnt,clk,q);
input cnt,clk;
output reg [31:0]q;
initial q<=32'd1;
always@(posedge clk)
begin
if (cnt)
begin
q[1]<=q[0];
q[2]<=q[1];
```

q[3]<=q[2];
q[4]<=q[3];
q[5]<=q[4];
q[6]<=q[5];
q[7]<=q[6];
q[8]<=q[7];
q[9]<=q[8];
q[10]<=q[9];
q[11]<=q[10];
q[12]<=q[11];
q[13]<=q[12];
q[14]<=q[13];
q[15]<=q[14];
q[16]<=q[15];
q[17]<=q[16];
q[18]<=q[17];
q[19]<=q[18];
q[20]<=q[19];
q[21]<=q[20];
q[22]<=q[21];
q[23]<=q[22];
q[24]<=q[23];
q[25]<=q[24];
q[26]<=q[25];
q[27]<=q[26];
q[28]<=q[27];
q[29]<=q[28];
q[30]<=q[29];

```
q[31]<=q[30];
q[0]<=q[31];
end
else
q <=64'b0;
end
endmodule
```

```
module ring_cnt_64b(cnt,clk,q);
input cnt,clk;
output reg [63:0]q;
initial q<=64'd1;
always@(posedge clk)
begin
if (cnt)
begin
q[1]<=q[0];
q[2]<=q[1];
q[3]<=q[2];
q[4]<=q[3];
q[5]<=q[4];
q[6]<=q[5];
q[7]<=q[6];
q[8]<=q[7];
q[9]<=q[8];
q[10]<=q[9];
q[11]<=q[10];
q[12]<=q[11];
```


q[13]<=q[12];
q[14]<=q[13];
q[15]<=q[14];
q[16]<=q[15];
q[17]<=q[16];
q[18]<=q[17];
q[19]<=q[18];
q[20]<=q[19];
q[21]<=q[20];
q[22]<=q[21];
q[23]<=q[22];
q[24]<=q[23];
q[25]<=q[24];
q[26]<=q[25];
q[27]<=q[26];
q[28]<=q[27];
q[29]<=q[28];
q[30]<=q[29];
q[31]<=q[30];
q[32]<=q[31];
q[33]<=q[32];
q[34]<=q[33];
q[35]<=q[34];
q[36]<=q[35];
q[37]<=q[36];
q[38]<=q[37];
q[39]<=q[38];
q[40]<=q[39];

```
q[41]<=q[40];
q[42]<=q[41];
q[43]<=q[42];
q[44]<=q[43];
q[45]<=q[44];
q[46]<=q[45];
q[47]<=q[46];
q[48]<=q[47];
q[49]<=q[48];
q[50]<=q[49];
q[51]<=q[50];
q[52]<=q[51];
q[53]<=q[52];
q[54]<=q[53];
q[55]<=q[54];
q[56]<=q[55];
q[57]<=q[56];
q[58]<=q[57];
q[59]<=q[58];
q[60]<=q[59];
q[61]<=q[60];
q[62]<=q[61];
q[63]<=q[62];
q[0]<=q[63];
end
else
q <=64'b0;
end
```

```
endmodule
```

```
module prescalar_ring_64b(cnt,clk,q);  
input cnt,clk;  
output [63:0]q;  
wire [1:0]q1;  
wire [7:0]q2;  
wire [63:0]q3;  
count_1b u1(cnt,clk,q[0]);  
ring_cnt_2b u2(cnt,clk,q1);  
count_2b u3(q1[1],clk,q[2:1]);  
ring_cnt_8b u4(cnt,clk,q2);  
count_3b u5(q2[7],clk,q[5:3]);  
ring_cnt_64b u6(cnt,clk,q3);  
count_58b u7(q3[63],clk,q[63:6]);  
endmodule
```

```
module mod_prescalar_ring_64b(cnt,clk,q);  
input cnt,clk;  
output [63:0]q;  
wire [3:0]q1;  
wire [15:0]q2;  
wire [31:0]q3;  
count_1b u1(cnt,clk,q[0]);  
ring_cnt_4b u2(cnt,clk,q1);  
count_2b u3(q1[3],clk,q[2:1]);  
ring_cnt_16b u4(cnt,clk,q2);  
count_3b u5(q2[15],clk,q[5:3]);
```

```
ring_cnt_32b u6(cnt,clk,q3);  
count_58b u7(q3[31],clk,q[63:6]);  
endmodule
```

```
module count_1b(en,clk,q);  
input en,clk;  
output reg q;  
initial q<=1'b0;  
always@(posedge clk)  
begin  
if(en)  
q<=q+1'b1;  
else  
q<=q;  
end  
endmodule
```

```
module count_2b(en,clk,q);  
input en,clk;  
output reg [1:0]q;  
initial q<=2'b0;  
always@(posedge clk)  
begin  
if(en)  
q<=q+2'b10;  
else  
q<=q;  
end  
end
```

```
endmodule
```

```
module count_3b(en,clk,q);
```

```
input en,clk;
```

```
output reg [2:0]q;
```

```
initial q<=3'b0;
```

```
always@(posedge clk)
```

```
begin
```

```
if(en)
```

```
q<=q+3'b011;
```

```
else
```

```
q<=q;
```

```
end
```

```
endmodule
```

```
module count_58b(en,clk,q);
```

```
input en,clk;
```

```
output reg [57:0]q;
```

```
initial q<=58'b0;
```

```
always@(posedge clk)
```

```
begin
```

```
if(en)
```

```
q<=q+58'd58;
```

```
else
```

```
q<=q;
```

```
end
```

```
endmodule
```